1. TOKENIZATION – TÁCH VĂN BẢN THÀNH TOKEN

HÀM / THÀNH PHẦN: tokenize(text: str) → list[str]

MỤC ĐÍCH: Tách văn bản tiếng Anh thành các token để mô hình xử lý.

CÁCH HOẠT ĐỘNG

- **Chữ thường:** dùng text.lower() để thống nhất các từ viết hoa/viết thường.
- **Biểu thức chính quy:** nhận diện chữ cái, số và một số từ có dấu nháy đơn.
- **Xử lý dấu câu:** bỏ dấu chấm, phẩy, !, $ … thay vì giữ chúng trong token.
- **Khác split():** không chỉ cắt văn bản tại các khoảng trắng.

VÍ DỤ / ĐOẠN MÃ

```
def tokenize(text: str):
    return re.findall(
        r"[a-z0-9]+(?:'[a-z]+)?",
        text.lower()
    )
```

Ví dụ: "Apple's New iPhone costs $999. Don't!"
→ ["apple's", "new", "iphone", "costs", "999", "don't"]

LƯU Ý • Đây là word-level regex tokenizer đơn giản; chưa phải tokenizer subword.

2. VOCABULARY – XÂY DỰNG BỘ TỪ VỰNG

HÀM / THÀNH PHẦN: build_vocab(train_rows, max_vocab=12000, min_freq=2)

MỤC ĐÍCH: Ánh xạ mỗi token được chọn từ tập TRAIN thành một ID số nguyên.

CÁCH HOẠT ĐỘNG

- **Counter:** đếm số lần mỗi token xuất hiện trong tập train.
- **max_vocab = 12000:** tối đa 12.000 mục từ, tính cả token đặc biệt.
- **min_freq = 2:** chỉ giữ từ xuất hiện ít nhất hai lần.
- **<pad> = 0:** dùng để đệm chuỗi; <unk> = 1 cho token ngoài vocabulary.
- **Tránh rò rỉ:** không xây dựng từ điển từ validation hoặc test.

VÍ DỤ / ĐOẠN MÃ

```
vocab = {"<pad>": 0, "<unk>": 1}
counts = Counter(
    t for row in train_rows
    for t in tokenize(row["text"])
)
# Giữ token phổ biến nhất
```

Ví dụ: "hanoi" → 37; token hiếm không có → 1 (<unk>).

LƯU Ý • 12.000 chỉ là siêu tham số cho bài trên lớp; bài về nhà dùng tối đa 20.000.

3. COLLATE_BATCH – GHÉP BATCH VÀ PADDING

HÀM / THÀNH PHẦN: collate_batch(samples)

MỤC ĐÍCH: Ghép các chuỗi có độ dài khác nhau thành tensor để huấn luyện theo mini-batch.

CÁCH HOẠT ĐỘNG

- **Đầu vào:** các mẫu (token_ids, label, text) từ Dataset.
- **Length:** ghi số token thật trước khi thêm <pad>.
- **Dynamic padding:** đệm bằng PAD_ID = 0 đến câu dài nhất trong batch.
- **Đầu ra:** ids [B,T], lengths [B], labels [B], texts.
- **Lợi ích:** giảm các vị trí đệm dư thừa so với cố định toàn bộ chuỗi.

VÍ DỤ / ĐOẠN MÃ

```
Câu A: [ 8, 15, 32, 21 ]
Câu B: [ 9, 18 ]

Sau collate_batch:
ids = [[8,15,32,21],
       [9,18, 0, 0]]
lengths = [4, 2]
```

Ví dụ: ids.shape = (2, 4); labels.shape = (2,).

LƯU Ý • Padding chỉ giúp tạo batch; lengths mới cho biết điểm kết thúc thật.

4. LSTMCLASSIFIER – KIẾN TRÚC PHÂN LOẠI

HÀM / THÀNH PHẦN: LSTMClassifier(...), forward(token_ids, lengths)

MỤC ĐÍCH: Chuyển một bản tin thành 4 logits của 4 chủ đề AG News.

CÁCH HOẠT ĐỘNG

- **Embedding:** chuyển ID thành vector 64 chiều.
- **LSTM một chiều:** xử lý chuỗi và tạo hidden state 96 chiều.
- **Linear:** ánh xạ hidden state thành 4 logits (chưa Softmax).
- **Forward:** Embedding → pack_padded_sequence → LSTM → Linear.
- **Đầu ra:** logits [B,4], dùng trực tiếp với CrossEntropyLoss.

VÍ DỤ / ĐOẠN MÃ

```
self.embedding = nn.Embedding(V, 64)
self.lstm = nn.LSTM(
    64, 96, batch_first=True
)
self.classifier = nn.Linear(96, 4)

# Forward
_, (h, _) = self.lstm(packed)
logits = self.classifier(h[-1])
```

Shapes: [B,T] → [B,T,64] → [B,96] → [B,4].

LƯU Ý • Các kích thước là ví dụ cấu hình mặc định; B,T thay đổi theo batch.

5. PACK_PADDED_SEQUENCE – BỎ QUA PADDING

HÀM / THÀNH PHẦN: pack_padded_sequence(embeddings, lengths, ...)

MỤC ĐÍCH: Cho LSTM xử lý đúng số bước thực tế của từng câu trong mini-batch.

CÁCH HOẠT ĐỘNG

- **Vấn đề:** câu ngắn được thêm <pad> đến độ dài chung của batch.
- **Không pack:** LSTM vẫn xử lý thêm các bước <pad> nếu truyền chuỗi đã đệm trực tiếp.
- **Có pack:** tính toán tuần hoàn chỉ trên các token thực theo lengths.
- **Hidden state:** h[-1] ứng với điểm kết thúc đúng của câu một chiều.
- **Hiệu quả:** có thể giảm tính toán thừa, đặc biệt khi độ dài chênh lệch lớn.

VÍ DỤ / ĐOẠN MÃ

```
packed = pack_padded_sequence(
    embeddings,
    lengths.cpu(),
    batch_first=True,
    enforce_sorted=False
)
_, (hidden_n, _) = lstm(packed)
```

"I love AI <pad> <pad>" → hidden state sau "AI", không sau <pad>.

LƯU Ý • Nếu dùng LSTM thông thường không pack, vẫn có thể lấy h đúng bằng length/mask.

6. TRAIN_ONE_EPOCH & EVALUATE – HUẤN LUYỆN VÀ ĐÁNH GIÁ

HÀM / THÀNH PHẦN: train_one_epoch(...), evaluate(..., collect_errors=False)

MỤC ĐÍCH: Tối ưu mô hình trên train; đo Loss, Accuracy, Macro-F1 trên validation/test.

CÁCH HOẠT ĐỘNG

- **Huấn luyện:** model.train() → zero_grad → forward → loss → backward → step.
- **Gradient clipping:** clip_grad_norm_(..., 1.0) hạn chế exploding gradient.
- **Đánh giá:** model.eval() và torch.no_grad() không cập nhật trọng số.
- **Metric:** tính loss trung bình, accuracy và macro-F1 trên toàn tập.
- **Phân tích lỗi:** collect_errors=True lưu câu, nhãn thật và nhãn sai.

VÍ DỤ / ĐOẠN MÃ

```
optimizer.zero_grad()
logits = model(ids, lengths)
loss = criterion(logits, labels)
loss.backward()
clip_grad_norm_(model.parameters(), 1.)
optimizer.step()

# evaluate: không backward
```

Báo cáo: train loss và val accuracy mỗi epoch; test sau khi chọn checkpoint.

LƯU Ý • Mô hình chọn bằng validation; test không dùng để điều chỉnh cấu hình.

7. SETTINGS VÀ EXPERIMENTS – THIẾT KẾ SO SÁNH

HÀM / THÀNH PHẦN: Settings, EXPERIMENTS, set_seed(seed)

MỤC ĐÍCH: Tổ chức các thực nghiệm công bằng, mỗi biến thể thay đúng một thành phần.

CÁCH HOẠT ĐỘNG

- **Settings:** cấu hình seed=42, max_len=100, batch_size=128, epochs=5, lr=0.001.
- **EXPERIMENTS:** khai báo 5 cấu hình gồm baseline, embedding, hidden, BiLSTM, Dropout.
- **set_seed(42):** khởi tạo ngẫu nhiên nhất quán để dễ đối chiếu kết quả.
- **Nguyên tắc:** cùng dữ liệu train/validation/test, optimizer và số epoch.
- **Không kết luận trước:** biến thể mạnh hơn chưa chắc đạt test accuracy cao hơn.

5 CẤU HÌNH TRONG NOTEBOOK

```
baseline:       emb=64,  hidden=96
embedding_128:  emb=128, hidden=96
hidden_128:     emb=64,  hidden=128
bilstm:         emb=64,  hidden=96,
                bidirectional=True
dropout_03:     emb=64,  hidden=96,
                dropout=0.3
```

Biến thể được so với baseline qua Accuracy, F1, Loss, số tham số, thời gian.

LƯU Ý • Trong file đáp án BTVN, max_vocab=20.000 (khác 12.000 của bài trên lớp).

8. AG NEWS FULL – TẢI, CHIA VÀ TẠO BATCH

HÀM / THÀNH PHẦN: load_agnews_full(), NewsDataset, collate_news()

MỤC ĐÍCH: Dùng toàn bộ AG News thật, tạo vocabulary chỉ từ train và padding theo batch.

CÁCH HOẠT ĐỘNG

- **load_agnews_full():** chia 120.000 mẫu train chính thức thành 114.000 train + 6.000 validation.
- **Official test:** giữ riêng 7.600 mẫu và chỉ dùng đánh giá cuối.
- **build_vocab():** tối đa 20.000 mục từ; chỉ đọc train.
- **NewsDataset:** lưu IDs, label, text và metadata (tỉ lệ UNK, có bị cắt không).
- **collate_news():** trả ids, lengths, labels và metadata để phân tích lỗi.

VÍ DỤ / ĐOẠN MÃ

```
train_rows, valid_rows, test_rows = \
    load_agnews_full(CFG)
vocab = build_vocab(train_rows, 20000, 2)
train_ds = NewsDataset(
    train_rows, vocab, CFG.max_len
)
loader = DataLoader(
    train_ds, collate_fn=collate_news
)
```

Quy tắc: không dùng validation/test để tạo vocabulary hay chọn cấu hình.

LƯU Ý • Nếu thiếu AG News thật, chương trình dừng; không tự chuyển sang dữ liệu giả.

9. EMBEDDING_DIM – THAY ĐỘ DÀI VECTOR TỪ

HÀM / THÀNH PHẦN: nn.Embedding(vocab_size, embedding_dim)

MỤC ĐÍCH: Thử xem biểu diễn mỗi token bằng vector nhiều chiều hơn có cải thiện phân loại không.

CÁCH HOẠT ĐỘNG

- **embedding_dim:** số phần tử trong vector của mỗi token.
- **Baseline:** embedding_dim=64 → mỗi token có vector 64 chiều.
- **Biến thể:** embedding_dim=128 → vector từ tăng lên 128 chiều.
- **Trong LSTM:** input_size bắt buộc khớp embedding_dim.
- **Đánh đổi:** nhiều tham số hơn, có thể học biểu diễn phong phú hơn nhưng tốn bộ nhớ.

VÍ DỤ / ĐOẠN MÃ

```
self.embedding = nn.Embedding(
    vocab_size, embedding_dim
)
self.lstm = nn.LSTM(
    input_size=embedding_dim,
    hidden_size=96,
    batch_first=True
)
```

Shapes: [B,T] → [B,T,64] hoặc [B,T,128].

LƯU Ý • Đổi embedding_dim không đồng nghĩa đổi số token trong vocabulary.

10. HIDDEN_DIM – THAY KÍCH THƯỚC TRẠNG THÁI ẨN

HÀM / THÀNH PHẦN: nn.LSTM(input_size=..., hidden_size=hidden_dim)

MỤC ĐÍCH: Thử khả năng biểu diễn của LSTM khi tăng số chiều hidden state.

CÁCH HOẠT ĐỘNG

- **hidden_dim:** số chiều vector trạng thái ẩn của LSTM.
- **Baseline:** hidden_dim=96 → mỗi bước tạo hidden state 96 chiều.
- **Biến thể:** hidden_dim=128 → hidden state 128 chiều.
- **Phải sửa Linear:** in_features = hidden_dim (với LSTM một chiều).
- **Đánh đổi:** mô hình tăng số tham số, thời gian và nguy cơ overfitting.

VÍ DỤ / ĐOẠN MÃ

```
self.lstm = nn.LSTM(
    input_size=64,
    hidden_size=hidden_dim,
    batch_first=True
)
self.classifier = nn.Linear(
    hidden_dim, 4
)
```

Ví dụ: hidden_dim=128 → hidden [B,128] → logits [B,4].

LƯU Ý • embedding_dim là chiều vector token; hidden_dim là chiều bộ nhớ LSTM.

11. BIDIRECTIONAL LSTM – ĐỌC CHUỖI HAI CHIỀU

HÀM / THÀNH PHẦN: nn.LSTM(..., bidirectional=True), torch.cat()

MỤC ĐÍCH: Ghép thông tin đọc từ trái sang phải và từ phải sang trái khi phân loại toàn câu.

CÁCH HOẠT ĐỘNG

- **LSTM thường:** chỉ khai thác chiều xuôi theo thứ tự token.
- **BiLSTM:** đồng thời xử lý chiều xuôi và chiều ngược.
- **Hai hidden cuối:** h[-2] đại diện chiều xuôi; h[-1] chiều ngược (1 layer).
- **torch.cat(...,dim=1):** ghép hai vector 96 chiều thành 192 chiều.
- **Lớp Linear:** nhận 2 × hidden_dim trước khi tạo 4 logits.

VÍ DỤ / ĐOẠN MÃ

```
self.lstm = nn.LSTM(
    64, 96, batch_first=True,
    bidirectional=True
)
_, (h, _) = self.lstm(packed)
features = torch.cat(
    (h[-2], h[-1]), dim=1
)
self.classifier = nn.Linear(192, 4)
```

Shapes: hai chiều [B,96] + [B,96] → [B,192] → [B,4].

LƯU Ý • BiLSTM có nhiều tham số hơn LSTM một chiều; phù hợp phân loại toàn văn bản.

12. DROPOUT – GIẢM PHỤ THUỘC VÀO ĐẶC TRƯNG

HÀM / THÀNH PHẦN: nn.Dropout(0.3), embedding_dropout, output_dropout

MỤC ĐÍCH: Giảm overfitting bằng cách vô hiệu hóa ngẫu nhiên một số phần tử khi huấn luyện.

CÁCH HOẠT ĐỘNG

- **Dropout = 0,3:** trong train đưa ngẫu nhiên khoảng 30% phần tử đầu ra về 0.
- **Trước LSTM:** embedding_dropout được áp dụng lên tensor Embedding.
- **Sau LSTM:** output_dropout áp dụng lên vector đặc trưng trước Linear.
- **Trong eval():** Dropout được tắt, không làm mất thông tin.
- **Số tham số:** không đổi; chỉ thay quá trình huấn luyện.

VÍ DỤ / ĐOẠN MÃ

```
self.embedding_dropout = (
    nn.Dropout(dropout)
)
self.output_dropout = (
    nn.Dropout(dropout)
)
embedded = self.embedding_dropout(
    self.embedding(token_ids)
)
logits = self.classifier(
    self.output_dropout(features)
)
```

Ví dụ: dropout=0.0 (baseline) so với dropout=0.3.

LƯU Ý • nn.LSTM(dropout=0.3) không tạo dropout giữa các tầng nếu num_layers=1.

13. HÀM HỖ TRỢ – SEED, THAM SỐ, THỜI GIAN

HÀM / THÀNH PHẦN: set_seed(), count_parameters(), cuda_sync()

MỤC ĐÍCH: Làm các lần chạy so sánh rõ ràng, có thể tái lập và đo chi phí tính toán.

CÁCH HOẠT ĐỘNG

- **set_seed(42):** đặt seed cho random, NumPy, PyTorch và CUDA nếu có.
- **count_parameters():** cộng số phần tử của trọng số có requires_grad=True.
- **cuda_sync():** chờ GPU thực thi xong trước/sau khi đo thời gian.
- **Thời gian/epoch:** trong lời giải là thời gian train + validation mỗi epoch.
- **Công bằng:** dùng cùng seed và điều kiện; kiểm tra nhiều seed nếu muốn kết luận chắc chắn.

VÍ DỤ / ĐOẠN MÃ

```
def count_parameters(model):
    return sum(
        p.numel()
        for p in model.parameters()
        if p.requires_grad
    )

def cuda_sync(device):
    if device.type == "cuda":
        torch.cuda.synchronize()
```

Ví dụ: BiLSTM tăng tham số; Dropout không thêm tham số học.

LƯU Ý • Thời gian đo phụ thuộc CPU/GPU và môi trường chạy, không chỉ kiến trúc.

14. HUẤN LUYỆN / ĐÁNH GIÁ TRONG BÀI VỀ NHÀ

HÀM / THÀNH PHẦN: train_one_epoch(...), evaluate(..., collect_errors=True)

MỤC ĐÍCH: Theo dõi validation mỗi epoch và thu đủ nhãn/dự đoán để báo cáo test.

CÁCH HOẠT ĐỘNG

- **train_one_epoch():** cập nhật embedding, LSTM và Linear bằng Adam + CrossEntropyLoss.
- **Gradient clipping:** giới hạn norm gradient ở 1.0 khi BPTT.
- **evaluate():** tắt gradient; tính loss, Accuracy, Macro-F1 trên toàn bộ dữ liệu.
- **collect_errors:** lưu text, nhãn đúng/sai, xác suất và thông tin token hóa.
- **Confusion matrix:** tạo từ nhãn thật và nhãn dự đoán, không từ loss.

VÍ DỤ / ĐOẠN MÃ

```
for epoch in range(1, epochs + 1):
    train_loss = train_one_epoch(...)
    val = evaluate(...)

# Sau khi chọn checkpoint:
test = evaluate(
    model, test_loader, loss_fn,
    device, collect_errors=True
)
```

Tập validation chọn checkpoint; test chỉ báo cáo cuối cùng.

LƯU Ý • CrossEntropyLoss nhận logits trực tiếp; Softmax chỉ phục vụ xác suất phân tích lỗi.

15. CHẠY THÍ NGHIỆM – CHỌN CHECKPOINT VÀ TEST

HÀM / THÀNH PHẦN: run_full_experiments(settings=CFG), torch.save/load

MỤC ĐÍCH: Chạy 5 cấu hình trên cùng AG News và chọn checkpoint theo validation loss.

CÁCH HOẠT ĐỘNG

- **Vòng for:** lần lượt chạy baseline và 4 biến thể trong EXPERIMENTS.
- **Mỗi epoch:** lưu train_loss, val_loss, val_accuracy, val_macro_f1, seconds.
- **Chọn checkpoint:** lưu trọng số khi validation loss giảm.
- **Chọn cấu hình:** tìm mô hình có best_validation_loss nhỏ nhất.
- **Test:** tải checkpoint từng cấu hình rồi báo cáo loss, accuracy, macro-F1.

VÍ DỤ / ĐOẠN MÃ

```
if validation["loss"] < best_loss:
    best_loss = validation["loss"]
    torch.save(
        model.state_dict(), checkpoint
    )

selected = min(
    records,
    key=lambda r: r["best_validation_loss"]
)["config"]
```

Đầu ra: best_*.pt, history_*.csv, so_sanh_test.csv, summary.json.

LƯU Ý • Không chọn best model bằng test accuracy; tránh bias đánh giá.

16. LEARNING CURVE VÀ CONFUSION MATRIX

HÀM / THÀNH PHẦN: plot_learning_curves(histories,...), plot_cm(cm,...)

MỤC ĐÍCH: Trực quan hóa tiến trình học và xác định lớp tin tức dễ bị dự đoán nhầm.

CÁCH HOẠT ĐỘNG

- **Learning curve:** vẽ train loss, validation loss và validation accuracy theo epoch.
- **So sánh:** đặt 5 cấu hình trên cùng biểu đồ để xem tốc độ hội tụ.
- **plot_cm():** vẽ ma trận 4×4 cho World, Sports, Business, Sci/Tech.
- **Cách đọc:** hàng = nhãn thật, cột = nhãn dự đoán; đường chéo là đúng.
- **Kiểm tra:** trace(cm) / sum(cm) phải trùng test accuracy.

VÍ DỤ / ĐOẠN MÃ

```
plot_learning_curves(
    histories, out_dir
)

cm = confusion_matrix(
    true_labels, predictions,
    labels=[0,1,2,3]
)
plot_cm(cm, output_path, name)
```

Ví dụ: World nhầm Business → xem lại các từ khóa về kinh tế quốc tế.

LƯU Ý • Các biểu đồ phải được tạo từ kết quả mô hình trên dữ liệu thật, không suy đoán số liệu.

17. PHÂN TÍCH ÍT NHẤT 5 CÂU DỰ ĐOÁN SAI

HÀM / THÀNH PHẦN: choose_diverse_errors(), write_error_analysis()

MỤC ĐÍCH: Chọn các ví dụ nhầm khác nhau và yêu cầu sinh viên giải thích dựa trên nội dung thật.

CÁCH HOẠT ĐỘNG

- **choose_diverse_errors():** ưu tiên các cặp nhãn thật → nhãn dự đoán khác nhau.
- **write_error_analysis():** tạo file markdown chứa tối đa 5 lỗi được chọn.
- **Thông tin kèm theo:** text gốc, dự đoán, xác suất, số token, tỉ lệ <unk>.
- **Dấu hiệu kỹ thuật:** tin bị truncate hoặc nhiều token ngoài vocabulary.
- **Phân tích ngữ nghĩa:** sinh viên tự đọc tin, giải thích nhầm lẫn và đề xuất cải thiện.

VÍ DỤ / ĐOẠN MÃ

```
chosen = choose_diverse_errors(
    errors, limit=5
)
write_error_analysis(
    errors,
    out_dir / "phan_tich_5_loi_test.md",
    settings.max_len
)
```

Ví dụ giả định: Business ↔ Sci/Tech khi tin nói về doanh thu công ty công nghệ.

LƯU Ý • Mã chỉ gợi ý dấu hiệu kỹ thuật; nguyên nhân ngữ nghĩa phải kiểm chứng bằng nội dung.